TechFix Express

Smart Web Experience: criando uma experiência digital com no-code, padrões web e IA

Parte teórica - Padrões Web para No Code e Low Code

Aluno: Alexandre Herbst | Data: 31/10/2026 | Site: https://alexandres-stupendous-site-c44438.webflow.io/

1. Problema, público e solução

Problema. Uma assistência técnica pequena de celulares e informática, sem equipe de desenvolvimento, perde clientes porque quem está com um aparelho quebrado quer saber, antes de sair de casa, o que pode ser o defeito, quanto custa e quanto tempo leva. Sem site, ou com um site que é só vitrine, essa resposta só vem por telefone ou indo até a loja, e muita gente desiste no caminho.

Público. Pessoas com celular ou notebook com defeito, de qualquer idade, que acessam principalmente pelo celular e preferem resolver pelo WhatsApp. O dono da loja também é público: precisa de algo que ele mesmo consiga manter, sem programador.

Solução. Um site de página única, para o cliente fictício TechFix Express, que apresenta os serviços com prazos e tem uma **triagem inteligente**: o visitante descreve o defeito e, em poucos segundos, recebe a categoria do problema, a urgência, uma faixa de preço, o prazo e os próximos passos, com um botão para continuar no WhatsApp. O diagnóstico final continua sendo do técnico na bancada.

2. Ferramenta escolhida e justificativa

A plataforma foi o **Webflow**, com automação no **Make** e inteligência artificial do **Google Gemini**. A tabela resume a escolha de cada peça.

Peça	Por que foi escolhida
Webflow	Permite montar o visual sem programar tudo do zero, mas deixa inserir HTML, CSS e JavaScript próprios. Isso atende ao pedido do trabalho de usar no-code e ainda personalizar com padrões web. O plano gratuito publica em um subdomínio, o que bastou para a entrega.
Make	Ferramenta visual de automação já conhecida por mim. O plano gratuito oferece webhook e módulo HTTP, suficientes para ligar o site à IA sem servidor próprio. Alternativas como o n8n exigiriam hospedagem.
Google Gemini	Tem cota gratuita e aceita pedir a resposta em um formato JSON fixo, o que facilita integrar com o site. O modelo usado foi o Gemini 3.1 Flash Lite, escolhido por responder em poucos segundos.

3. Como HTML, CSS e JavaScript foram usados

A personalização foi o centro do trabalho. Em vez de componentes prontos de template, as seções foram escritas com HTML e CSS próprios e inseridas no Webflow.

HTML semântico. O site usa `header`, `nav`, `section`, `footer`, listas para os cards e `details/summary` para o FAQ, que abre e fecha sem JavaScript. Há um link "pular para o conteúdo" no topo.

CSS. Cada elemento recebe uma classe com prefixo `tf-`. As cores, espaçamentos e a hierarquia visual foram definidos por mim. Os grids usam `repeat(auto-fit, minmax(...))` e a tipografia usa `clamp()`, de modo que o layout se ajusta à largura da tela sem depender de pontos de quebra fixos. O construtor de HTML do Webflow aceita só seletores de classe simples e ignora `@media`, o que levou a essa técnica.

JavaScript. O formulário da triagem fica em dois blocos HTML Embed (o limite de tamanho do Webflow é de cerca de 10 mil caracteres por bloco). O script faz:

- validação dos campos com mensagens de erro ligadas a cada campo e contador de caracteres;
- envio dos dados com `fetch`, no formato de formulário (URLSearchParams), que evita a verificação prévia de CORS no webhook do Make;
- limite de tempo de 20 segundos com `AbortController`;
- um **plano B**: se a IA falhar ou demorar, o site classifica o problema por palavras-chave e avisa que é um resultado de demonstração;
- escape do texto recebido da IA antes de colocá-lo na página, para evitar injeção de HTML.

Estrutura visual do código

As figuras abaixo mostram como as três linguagens se encaixam no projeto. A Figura 1 é a árvore da página: as seções em azul foram escritas em HTML e CSS dentro do Webflow, e a seção da triagem recebe dois blocos de embed, um com HTML e CSS (verde) e outro com JavaScript (laranja).



Figura 1 - Árvore HTML da página única e onde ficam os embeds da triagem.

A Figura 2 resume a divisão de responsabilidades. O HTML define o conteúdo e a estrutura; o CSS cuida da aparência e da adaptação ao tamanho da tela; o JavaScript controla o comportamento. O CSS encontra os elementos por classe e id, e o JavaScript os encontra pelos ids do formulário para

ler os campos, mostrar erros e escrever o resultado.

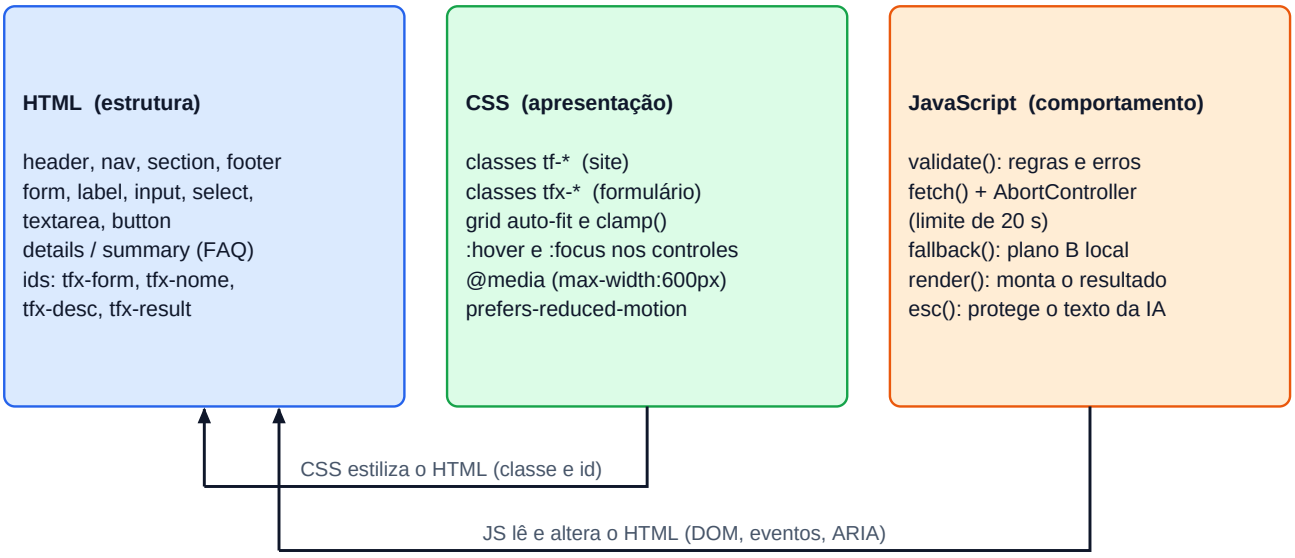


Figura 2 - As três camadas (HTML, CSS e JavaScript) e como elas se relacionam.

A Figura 3 mostra o que acontece quando o visitante envia o formulário, passando pelo navegador, pelo Make e pelo Gemini, e o plano B que garante uma resposta mesmo quando a IA falha.

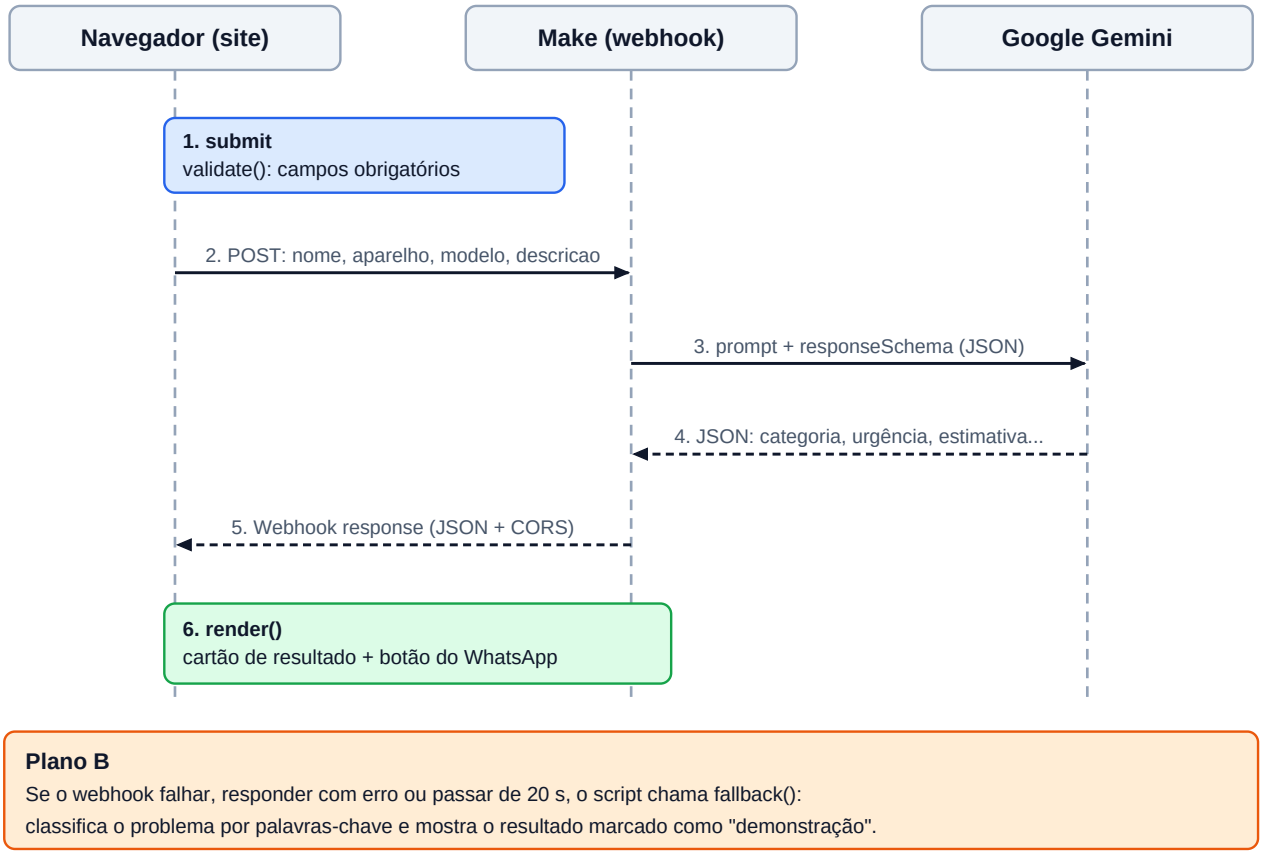


Figura 3 - Fluxo de execução do JavaScript, do envio do formulário até o resultado na tela.

4. Recurso de IA e automação

O fluxo tem três módulos no Make:

Módulo	Função
1. Custom webhook	Recebe nome, tipo de aparelho, modelo e descrição do defeito, enviados pelo site.
2. HTTP	Chama a API do Gemini com um prompt que define o papel de assistente de triagem e com um esquema de resposta (responseSchema) que obriga a devolver um JSON com categoria, urgência (alta, media ou baixa), estimativa mínima e máxima, prazo, resumo e próximos passos.
3. Webhook response	Devolve esse JSON ao site, com Content-Type application/json e o cabeçalho de CORS liberado.

A **chave da API do Gemini fica somente no Make**, nunca no código do site. Como o formato de resposta é fixo, o site consegue montar o resultado sem quebrar. O prompt pede estimativas, não certezas, e o resultado traz o aviso de que o valor final depende da avaliação técnica. Durante o desenvolvimento, em testes de ponta a ponta, a resposta completa levou poucos segundos.

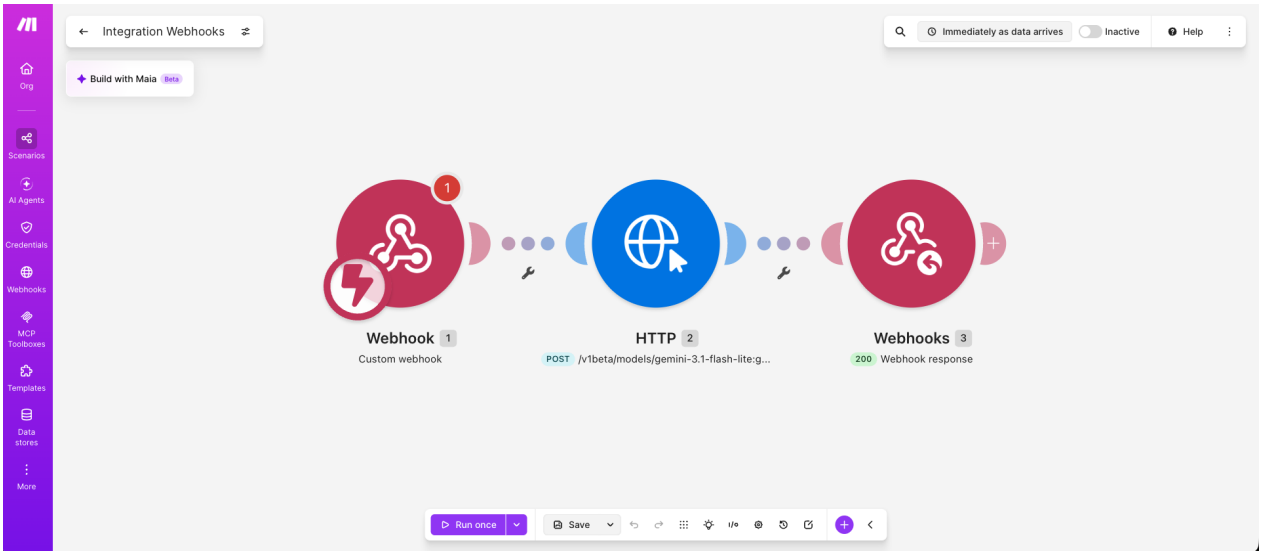


Figura 4 - Cenário no Make: Custom webhook, HTTP (chamada ao Gemini) e Webhook response.

O histórico de execuções do cenário confirma o funcionamento: as chamadas feitas a partir do site (gatilho "Instant") terminaram com sucesso em cerca de 1 a 3 segundos. As falhas que aparecem mais abaixo no histórico são dos testes iniciais, feitos com modelos do Gemini indisponíveis ou sobrecarregados.

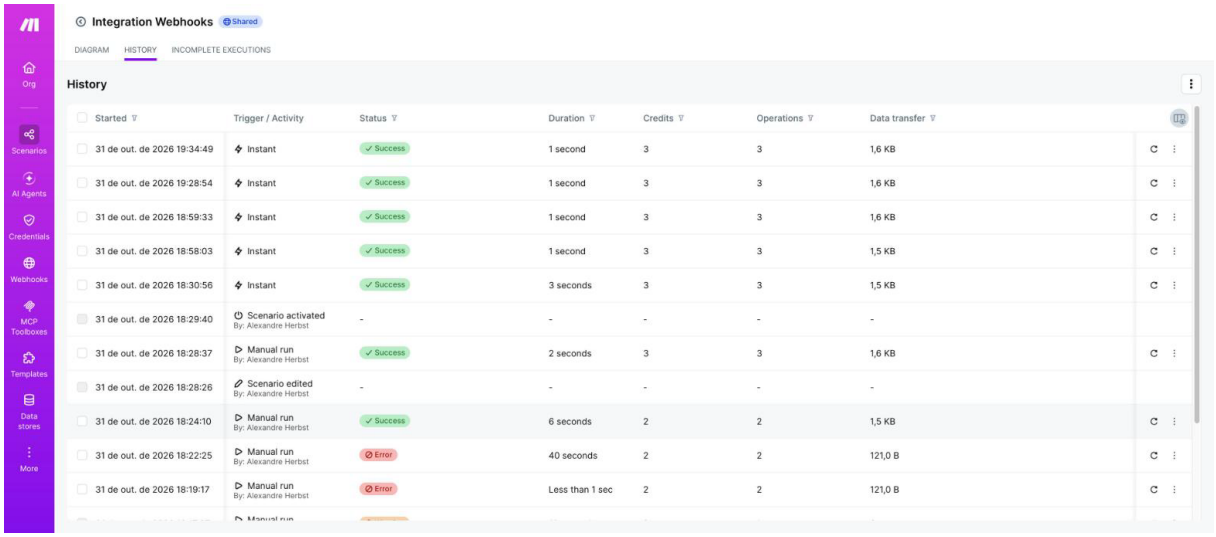


Figura 5 - Histórico de execuções do cenário no Make, com execuções bem-sucedidas.

No site publicado, o resultado da triagem aparece com a categoria, a urgência, o prazo, a estimativa, o resumo, os próximos passos e o botão para continuar no WhatsApp, junto com o aviso de que a análise foi gerada por IA e a estimativa é aproximada.

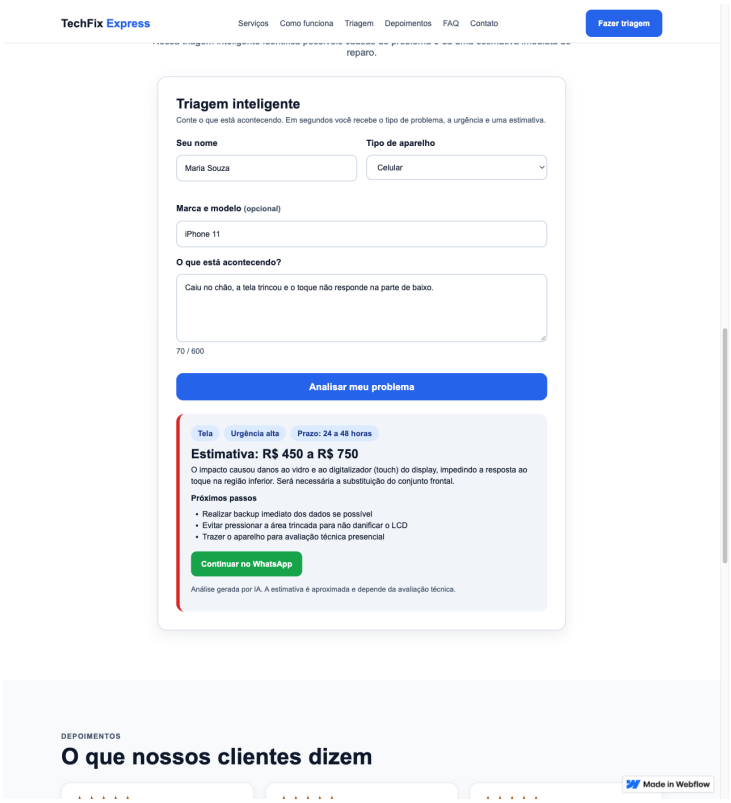


Figura 6 - Triagem preenchida e resultado gerado pela IA no site publicado.

5. Responsividade e acessibilidade

Responsividade. Os grids se reorganizam sozinhos conforme a largura, as fontes e os espaçamentos escalam com `clamp()`, o menu quebra para uma segunda linha em telas estreitas e o formulário passa para uma coluna em telas de até 600 px. Esse comportamento está no código. Em teste com a janela do navegador a 390 px de largura, a página não apresentou rolagem horizontal (Figura 7).



Figura 7 - Topo do site em uma tela de 390 px de largura (celular).

Acessibilidade.

- cada campo do formulário tem rótulo associado e mensagem de erro ligada por `aria-describedby`;
- campos com erro recebem `aria-invalid` e o foco vai para o primeiro deles;
- o resultado da IA aparece em uma região `aria-live`, anunciada por leitores de tela;
- o foco do teclado é visível nos controles e links;
- animações respeitam `prefers-reduced-motion`;
- as cores foram escolhidas com bom contraste, e os botões têm tamanho adequado para toque.

6. Limitações e aprendizados

Limitações.

- A estimativa é aproximada. A IA não examina o aparelho, e o valor final depende do técnico.
- A URL do webhook fica visível no código da página, como em qualquer site que chama um webhook. Em produção, seria preciso limitar o uso (por exemplo, com limite de requisições).
- O Make e o Gemini têm cotas gratuitas; ao excedê-las, o site cai no modo de demonstração.
- Para caber no plano gratuito, o site tem uma só página e não usa CMS.
- Cliente, depoimentos, e-mail e prazos dos serviços são fictícios.

Aprendizados.

- Plataformas no-code aceleram, mas têm limites (de plano, de recursos e de CSS) que só aparecem na prática; saber HTML e CSS permitiu contorná-los.

- A IA gerada pela própria plataforma cometeu erros (criou CMS que bloqueou a publicação e deixou conteúdo de template), por isso é preciso revisar o que ela entrega.
- Integrar uma IA a um site exige um formato de resposta fixo, limite de tempo e um plano B. Os modelos podem ficar indisponíveis, e na prática um deles falhou por excesso de demanda durante os testes.
- Manter segredos (a chave da API) fora do código do site, no serviço de automação, foi a decisão de segurança mais importante do projeto.